

Lab Assignment 8

Object-Oriented Programming

Branch

B.E., 2nd Year – ENC



Submitted By:

Kaushtubh Kumar

Roll No. 1024150191

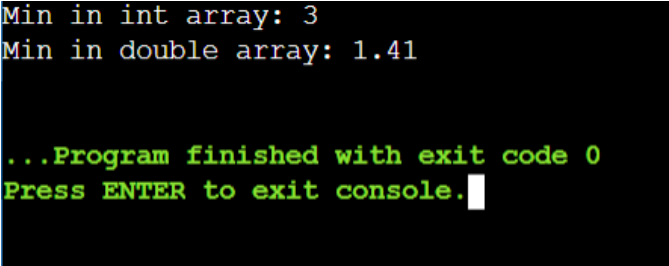
Batch: 2023

Function Templates

1. Write a function template to swap two variables of any data type.

Code	Testcase
<pre>#include <iostream> using namespace std; template <typename T> void swapValues(T &a, T &b) { T temp = a; a = b; b = temp; } int main() { int x = 5, y = 10; cout << "Before swap: x=" << x << ", y=" << y << endl; swapValues(x, y); cout << "After swap: x=" << x << ", y=" << y << endl; double a = 3.14, b = 2.71; cout << "Before swap: a=" << a << ", b=" << b << endl; swapValues(a, b); cout << "After swap: a=" << a << ", b=" << b << endl; return 0; }</pre>	<pre>Before swap: x=5, y=10 After swap: x=10, y=5 Before swap: a=3.14, b=2.71 After swap: a=2.71, b=3.14 ...Program finished with exit code 0 Press ENTER to exit console.</pre>

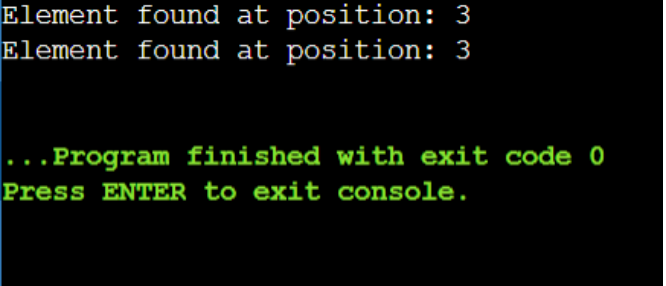
2. Write a function template to find the minimum element in an array.

Code	Testcase
<pre>#include <iostream> using namespace std; template <typename T> T findMin(T arr[], int size) { T min = arr[0]; for (int i = 1; i < size; i++) { if (arr[i] < min) { min = arr[i]; } } return min; } int main() { int intArr[] = {45, 12, 78, 3, 89, 25}; cout << "Min in int array: " << findMin(intArr, 6) << endl; double doubleArr[] = {3.14, 2.71, 1.41, 5.67}; cout << "Min in double array: " << findMin(doubleArr, 4) << endl; return 0; }</pre>	 <pre>Min in int array: 3 Min in double array: 1.41 ...Program finished with exit code 0 Press ENTER to exit console.</pre>

3. Write a function template to sort an array using bubble sort.

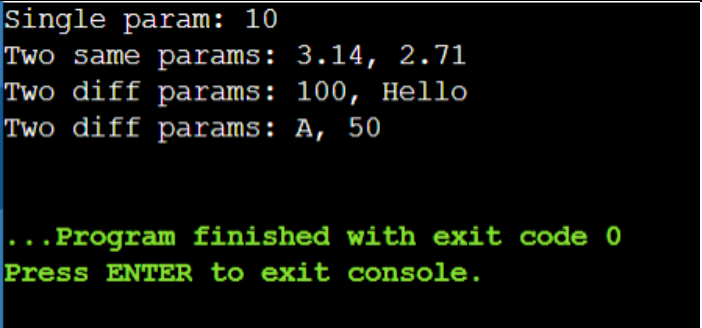
Code	Testcase
<pre>#include <iostream> using namespace std; template <typename T> void bubbleSort(T arr[], int size) { for (int i = 0; i < size - 1; i++) { for (int j = 0; j < size - i - 1; j++) { if (arr[j] > arr[j + 1]) { T temp = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = temp; } } } } template <typename T> void display(T arr[], int size) { for (int i = 0; i < size; i++) cout << arr[i] << " "; cout << endl; } int main() { int intArr[] = {64, 34, 25, 12, 22, 11, 90}; cout << "Before sorting: "; display(intArr, 7); bubbleSort(intArr, 7); cout << "After sorting: "; display(intArr, 7); return 0; }</pre>	<pre>Before sorting: 64 34 25 12 22 11 90 After sorting: 11 12 22 25 34 64 90 ...Program finished with exit code 0 Press ENTER to exit console.</pre>

4. Write a function template to perform linear search.

Code	Testcase
<pre>#include <iostream> using namespace std; template <typename T> int linearSearch(T arr[], int size, T key) { for (int i = 0; i < size; i++) { if (arr[i] == key) { return i; } } return -1; } int main() { int intArr[] = {10, 25, 35, 45, 55}; int pos = linearSearch(intArr, 5, 35); if (pos != -1) cout << "Element found at position: " << pos + 1 << endl; else cout << "Element not found" << endl; char charArr[] = {'A', 'B', 'C', 'D'}; pos = linearSearch(charArr, 4, 'C'); if (pos != -1) cout << "Element found at position: " << pos + 1 << endl; return 0; }</pre>	 Element found at position: 3 Element found at position: 3 ...Program finished with exit code 0 Press ENTER to exit console.

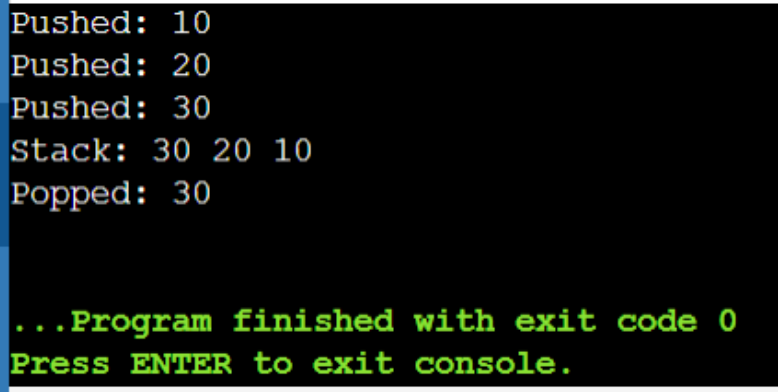
5. Design overloaded versions of a function template process() such that:

- i) Accepts a single parameter
- ii) Accepts two parameters of the same type
- iii) Accepts two parameters of different types

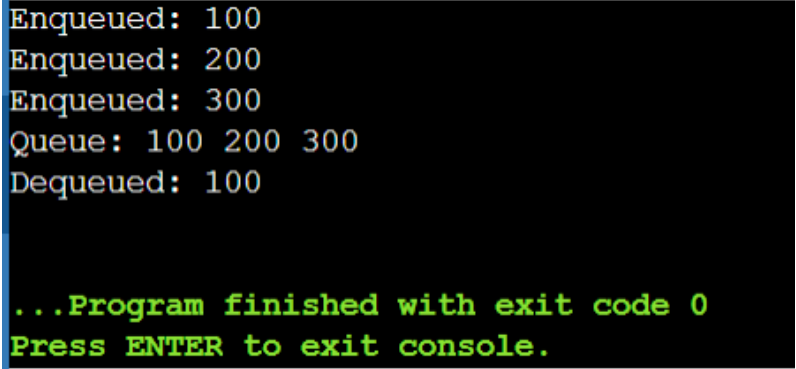
Code	Testcase
<pre>#include <iostream> using namespace std; // Single parameter template <typename T> void process(T a) { cout << "Single param: " << a << endl; } // Two parameters of same type template <typename T> void process(T a, T b) { cout << "Two same params: " << a << ", " << b << endl; } // Two parameters of different types template <typename T, typename U> void process(T a, U b) { cout << "Two diff params: " << a << ", " << b << endl; } int main() { process(10); process(3.14, 2.71); process(100, "Hello"); process('A', 50); return 0; }</pre>	 Single param: 10 Two same params: 3.14, 2.71 Two diff params: 100, Hello Two diff params: A, 50 ...Program finished with exit code 0 Press ENTER to exit console.

Class Templates

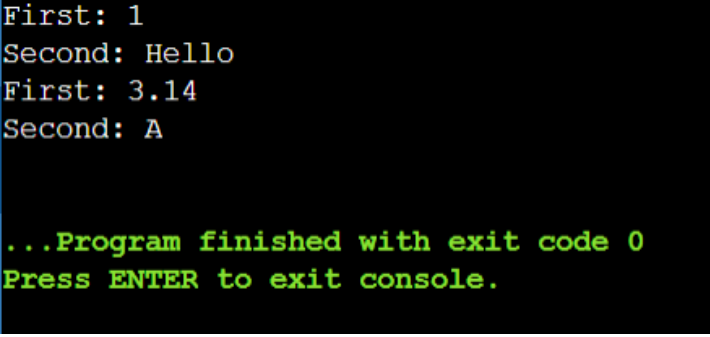
6. Develop a class template for stack with push and pop operations.

Code	Testcase
<pre>#include <iostream> using namespace std; template <typename T> class Stack { private: T* arr; int top; int capacity; public: Stack(int size = 10) { arr = new T[size]; top = -1; capacity = size; } ~Stack() { delete[] arr; } void push(T value) { if (top >= capacity - 1) { cout << "Stack Overflow" << endl; return; } arr[++top] = value; cout << "Pushed: " << value << endl; } T pop() { if (top < 0) { cout << "Stack Underflow" << endl; return T(); } return arr[top--]; } void display() { for (int i = top; i >= 0; i--) cout << arr[i] << " "; cout << endl; } }; int main() { Stack<int> intStack; intStack.push(10); intStack.push(20); intStack.push(30); cout << "Stack: "; intStack.display(); cout << "Popped: " << intStack.pop() << endl; return 0; }</pre>	

7. Develop a class template for queue with enqueue and dequeue operations.

Code	Testcase
<pre>#include <iostream> using namespace std; template <typename T> class Queue { private: T* arr; int front, rear; int capacity; public: Queue(int size = 10) { arr = new T[size]; front = 0; rear = -1; capacity = size; } ~Queue() { delete[] arr; } void enqueue(T value) { if (rear >= capacity - 1) { cout << "Queue Overflow" << endl; return; } arr[++rear] = value; cout << "Enqueued: " << value << endl; } T dequeue() { if (front > rear) { cout << "Queue Underflow" << endl; return T(); } return arr[front++]; } void display() { for (int i = front; i <= rear; i++) cout << arr[i] << " "; cout << endl; } }; int main() { Queue<int> q; q.enqueue(100); q.enqueue(200); q.enqueue(300); cout << "Queue: "; q.display(); cout << "Dequeued: " << q.dequeue() << endl; return 0; }</pre>	

8. Create a class template to store and display a pair of values.

Code	Testcase
<pre>#include <iostream> using namespace std; template <typename T1, typename T2> class Pair { private: T1 first; T2 second; public: Pair(T1 f, T2 s) { first = f; second = s; } void display() { cout << "First: " << first << endl; cout << "Second: " << second << endl; } T1 getFirst() { return first; } T2 getSecond() { return second; } }; int main() { Pair<int, string> p1(1, "Hello"); p1.display(); Pair<double, char> p2(3.14, 'A'); p2.display(); return 0; }</pre>	 A screenshot of a terminal window showing the output of the program. The output consists of two lines: "First: 1" followed by "Second: Hello" on the next line, and "First: 3.14" followed by "Second: A" on the next line. Below this, there is a green text message: "...Program finished with exit code 0" followed by "Press ENTER to exit console." on the next line. <pre>First: 1 Second: Hello First: 3.14 Second: A ...Program finished with exit code 0 Press ENTER to exit console.</pre>

9. Create a class template for basic arithmetic operations.

Code	Testcase
<pre>#include <iostream> using namespace std; template <typename T> class Arithmetic { private: T a, b; public: Arithmetic(T x, T y) { a = x; b = y; } T add() { return a + b; } T subtract() { return a - b; } T multiply() { return a * b; } T divide() { if (b != 0) return a / b; else { cout << "Division by zero!" << endl; return T(); } } void display() { cout << "a=" << a << ", b=" << b << endl; cout << "Add: " << add() << endl; cout << "Sub: " << subtract() << endl; cout << "Mul: " << multiply() << endl; cout << "Div: " << divide() << endl; } }; int main() { Arithmetic<int> calc1(10, 5); calc1.display(); Arithmetic<double> calc2(20.5, 4.0); calc2.display(); return 0; }</pre>	<pre>a=20.5, b=4 Add: 24.5 Sub: 16.5 Mul: 82 Div: 5.125 ...Program finished with exit code 0 Press ENTER to exit console.</pre>

10. Create a class template to input and display array elements.

Code	Testcase
<pre>#include <iostream> using namespace std; template <typename T> class Array { private: T* arr; int size; public: Array(int s) { size = s; arr = new T[size]; } ~Array() { delete[] arr; } void input() { cout << "Enter " << size << " elements: "; for (int i = 0; i < size; i++) cin >> arr[i]; } void display() { cout << "Array elements: "; for (int i = 0; i < size; i++) cout << arr[i] << " "; cout << endl; } }; int main() { Array<int> intArr(5); cout << "For Integer Array:" << endl; intArr.input(); intArr.display(); Array<double> doubleArr(3); cout << "\nFor Double Array:" << endl; doubleArr.input(); doubleArr.display(); return 0; }</pre>	<pre>For Integer Array: Enter 5 elements: 45 23 623 53 12 Array elements: 45 23 623 53 12 For Double Array: Enter 3 elements: 542 34 19 Array elements: 542 34 19 ...Program finished with exit code 0 Press ENTER to exit console.</pre>